

COMP 3069

Computer Graphics



Lecture 6:
Spaces
Autumn 2025



The University of
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

Contents

- ◆ Transform a vertex in 3-D space through six spaces to 2-D
- ◆ Define matrices used to transform between the spaces
- ◆ Use perspective division to transform from clip space to normalised device coordinate (NDC) space
- ◆ Visualise triangles constructed from vertices in screen space



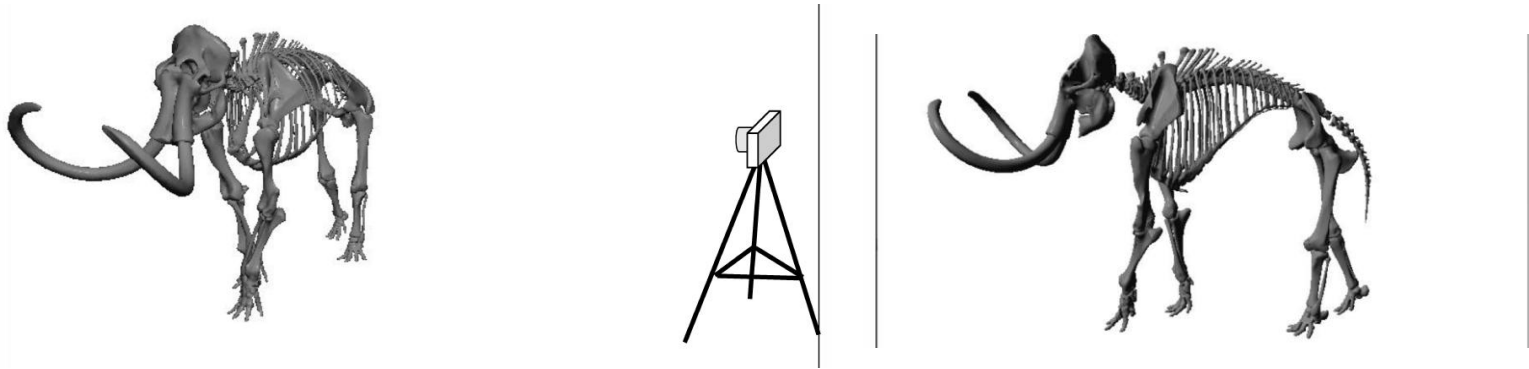
Requirements for 3D Rendering

- ◆ Triangles specified by vertices 用顶点定义的三角形
- ◆ A camera, for which the programmer will specify the location 相机（要给出位置）
- ◆ A display to draw it all on 显示设备



Projection from 3D to 2D 从三维空间到二维空间的投影

- ◆ An important part of rendering is **projection from 3D to 2D** 渲染的重要部分之一就是从小三维到二维的投影。
- ◆ The object, also called **model**, is made up of **triangles** 物体（也叫模型）是由三角形组成的。



Transformations between Spaces

空间之间的变换

◆ Model transform 模型变换

把顶点从模型空间 (Model Space) 转换到世界空间 (World Space)。 $v_{world} = M * v_{model}$

- Transformation of vertices from model space to world space

◆ View transform 观察变换

把顶点从世界空间转换到相机空间 (View Space / Eye Space)。 $v_{view} = V * v_{world}$

- Transformation of vertices from world space to view space

◆ Projection transform 投影变换

把顶点从视图空间转换到裁剪空间 (Clip Space) $v_{clip} = P * v_{view}$

- Projection of vertices from view space to clip space

$$(x_{ndc}, y_{ndc}, z_{ndc}) = \left(\frac{x_{clip}}{w}, \frac{y_{clip}}{w}, \frac{z_{clip}}{w} \right)$$

前一步的结果是齐次坐标 (x, y, z, w) ；

透视除法通过计算 $(x/w, y/w, z/w)$ 得到 NDC 空间坐标 (Normalized Device Coordinates)。

在 NDC 中，坐标范围都是 $[-1, 1]$ ，这样 GPU 可以统一处理。

直观理解：w 越大（物体越远），除以 w 之后得到的 x、y 越小，远的物体看起来更小（透视感）。

◆ Perspective Division 透视除法

用每个顶点的同类分量 (w) 去除以它的 x, y, z

- Division of each component by its homogeneous component

◆ Viewport Transformation 视口变换

把标准化坐标 (NDC) 映射到屏幕像素坐标。

- Mapping of normalized coordinates to vertices at screen pixel coordinates

NDC 的范围是 $[-1, 1]$ ；

屏幕的像素范围是 $[0, width] \times [0, height]$ ；

视口变换把 NDC 转换到屏幕像素坐标系，并决定绘制区域。

直观理解：左下角 $(-1, -1)$ 屏幕 $(0, 0)$ ，右上角 $(1, 1)$ 屏幕 $(width, height)$

$$x_{screen} = \frac{width}{2} (x_{ndc} + 1)$$

$$v_{screen} = Viewport \times PerspectiveDiv \times Projection \times View \times Model \times v_{model}$$

或者更常见写法 (矩阵乘法顺序):

$$v_{clip} = P \cdot V \cdot M \cdot v_{model}$$

$$v_{ndc} = v_{clip} / w$$

$$v_{screen} = Viewport(v_{ndc})$$

$$y_{screen} = \frac{height}{2} (y_{ndc} + 1)$$

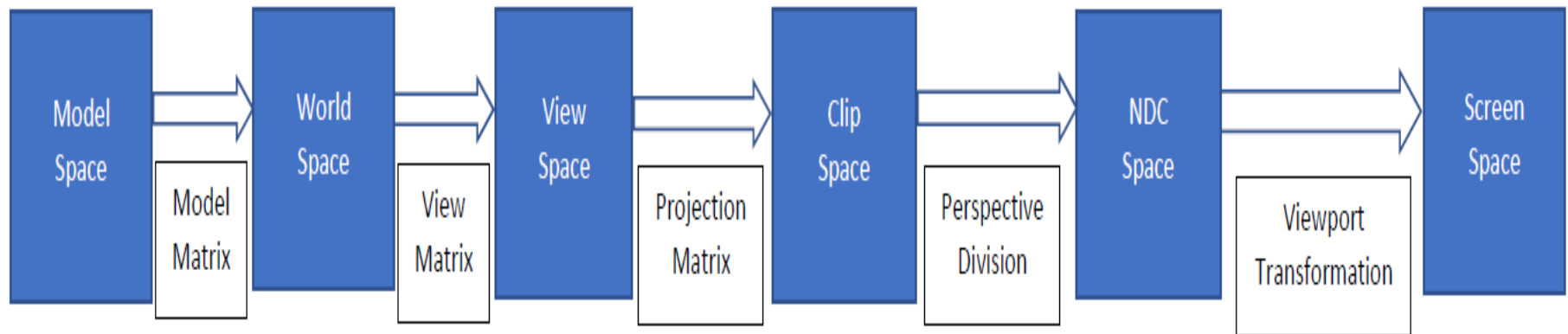


The University of
Nottingham

Spaces & Transformations

- ◆ Vertices are transformed through six spaces before being used to construct triangles for rasterisation

每个顶点会依次经过六个空间的转换，然后才被用来构建三角形进行光栅化（rasterisation）。
每一块蓝色方框代表一种坐标空间（Space），每个白色框代表对应的变换矩阵或操作（Transform）。
这些变换依次发生在 GPU 渲染管线的“顶点处理阶段”。



Model Space

模型空间是针对单个模型的局部坐标系，这个模型由顶点（vertices）组成

- ◆ **Model Space** is relative to an individual model, which is made up of vertices
立方体的每个角都是一个顶点，这些顶点的位置都是相对于立方体中心定义的。
- ◆ **A cube** is made up of one vertex at each corner, which are positioned relative to the centre of the cube
在模型空间中，没有任何关于模型在世界中的位置或方向的信息。
- ◆ In model space, there is no information about where a model is relative to anything in the world
三维几何是由顶点构建出来的。
- ◆ **3D geometry** can be modelled from vertices

模型空间 = 模型的“内部坐标系”，描述模型本身的形状。

- 原点通常在模型中心；
- 顶点坐标相对模型定义；
- 不含任何关于“世界位置”的信息；
- 后续用 Model Matrix 把它放进 World Space。

 可视化解：

概念	含义	是否含位置信息	坐标原点	举例
Model Space	模型自身的坐标系	✗ 无世界位置信息	模型中心	立方体以 (0,0,0) 为中心建模
World Space	整个场景的坐标系	✓ 有世界位置	场景中心	把立方体放在 (10,0,-5)



Model Space

模型空间（或称局部空间）是相对于单个模型自身的坐标系。

- ◆ Model space (local space) is relative to an individual model

通常，模型空间的原点在模型的中心。

- ◆ Usually, the origin of the model space is at the centre of the model

- 绿色箭头: Y 轴 (向上)
- 红色箭头: X 轴 (向右)
- 蓝色箭头: Z 轴 (向前或向外)

三条轴交于原点 $(0, 0, 0)$ ，表示模型空间的原点在立方体中心。

图中的顶点 $v1 = (-1, 1, -1)$ 表示:

- 沿 X 轴向左 1 个单位;
- 沿 Y 轴向上 1 个单位;
- 沿 Z 轴向后 (负方向) 1 个单位。

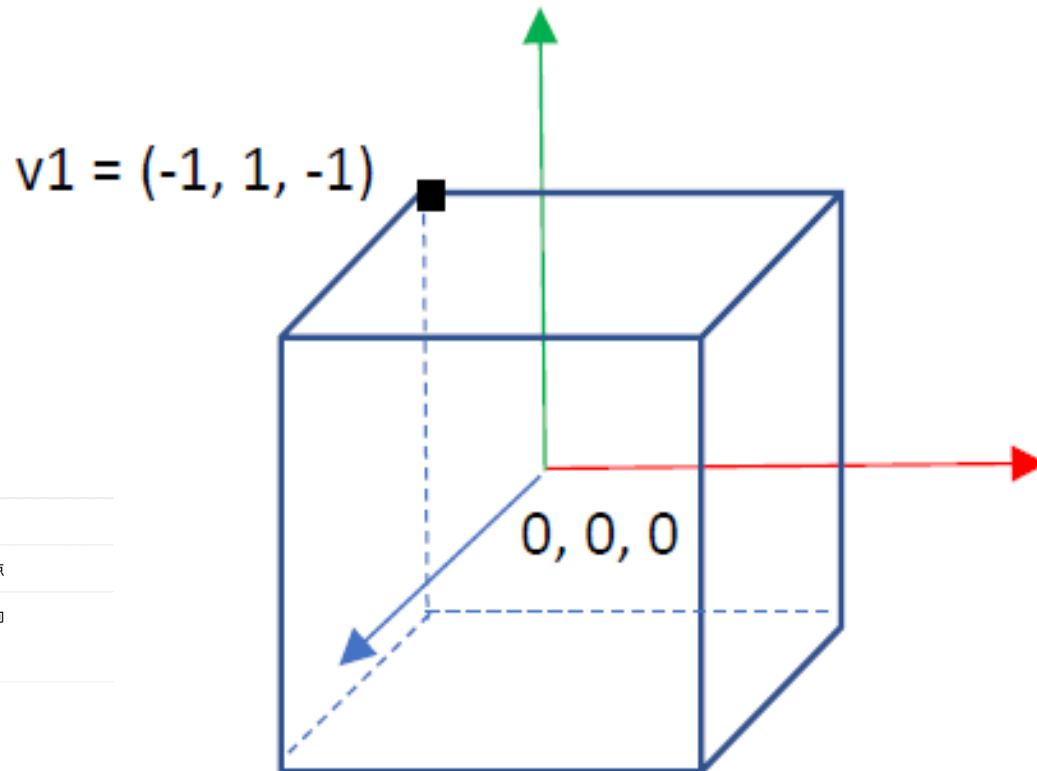
■ 换句话说:

这是一个立方体左上后角的顶点坐标。所有顶点都是以立方体中心为参考定义的。

维度	在模型空间中代表什么	是否与世界有关
X, Y, Z	模型顶点相对模型中心的位置	✗ 无关
(0,0,0)	模型的中心点	✗ 不是世界原点
单位方向	模型的局部坐标轴	✗ 不是世界方向

■ 总结一句话:

模型空间描述模型自己的形状与局部坐标系，原点在模型中心，所有顶点坐标都是相对这个中心定义的。



World Space

世界空间是相对于一个更大的坐标系统的。

- ◆ World Space is relative to a larger coordinate system

模型的顶点不是直接在世界空间中建模的。

- ◆ Model vertices are not directly modeled in world space

顶点首先定义在模型空间中，然后被移动（变换）到世界空间中的合适位置。

- ◆ The vertices are positioned in model space, and then moved to the appropriate places in the world space

一句话总结：

世界空间是整个场景的公共坐标系统。模型在自己的局部空间中定义顶点，然后通过模型矩阵移动到世界空间的合适位置。



World Space

世界空间用于在场景中放置所有模型的位置。

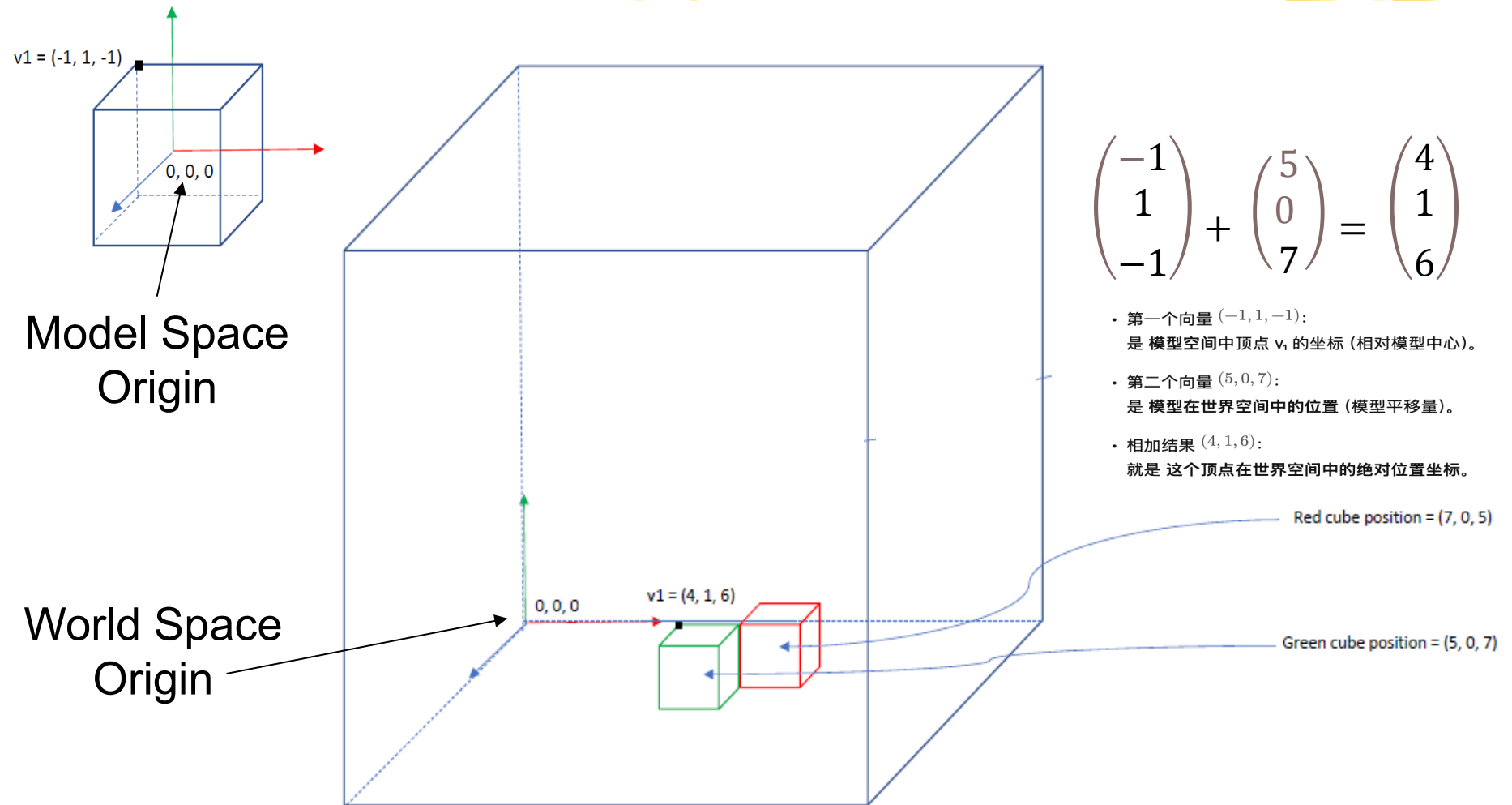
- ◆ World space is used for positioning all the models in the scene

世界坐标系的原点可以放在任意位置，但通常放在空间中心或某个角落。

- ◆ The origin can be at any point in the space but tends to be in the centre of the space or at a corner



World Space



Model Space -> World Space

Model Matrix

每个模型都会构造一个模型矩阵，这个矩阵由旋转和平移（有时也包含缩放）组合而成。

- ◆ A model matrix is constructed for each of the models by combining rotation and translation
模型矩阵会变换该模型的所有顶点。
- ◆ The model matrix transforms all of the vertices specifying that model
绿色立方体的模型矩阵以及它应用到一个顶点的过程如下
- ◆ The model matrix for the green cube and its application to a vertex are

$$M = \begin{pmatrix} 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 7 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad u = M \times v = \begin{pmatrix} 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 7 \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} -1 \\ 1 \\ -1 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 1 \\ 6 \\ 1 \end{pmatrix}$$



View Space

观察空间是从观察者（viewer，也就是相机）的视角出发的坐标系，也叫“相机空间”或“眼空间”。

- ◆ View Space has all vertices from the perspective of the viewer. It is also known as camera space or eye space.

观察空间代表从相机视角看到的场景；此时相机位于坐标原点 $(0,0,0)$ ，并朝着 $-Z$ 方向看。

- ◆ It represents the scene from the perspective of the camera, where the camera is positioned at the origin looking along the negative z-axis.

顶点最初在模型空间中定义，被移动到世界空间后，再移动到相机视角下的相对位置。

- ◆ Vertices positioned in model space, moved to the appropriate places in the world, and then moved to the positions relative to the viewer.



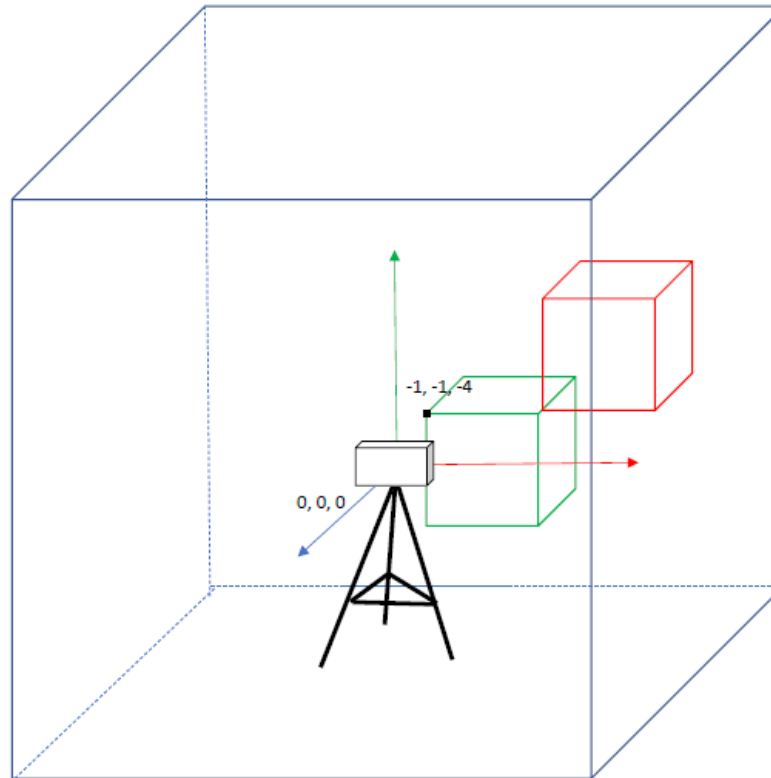
View Space

观察空间中的每个顶点，都是从相机（观察者）的视角来表示的。

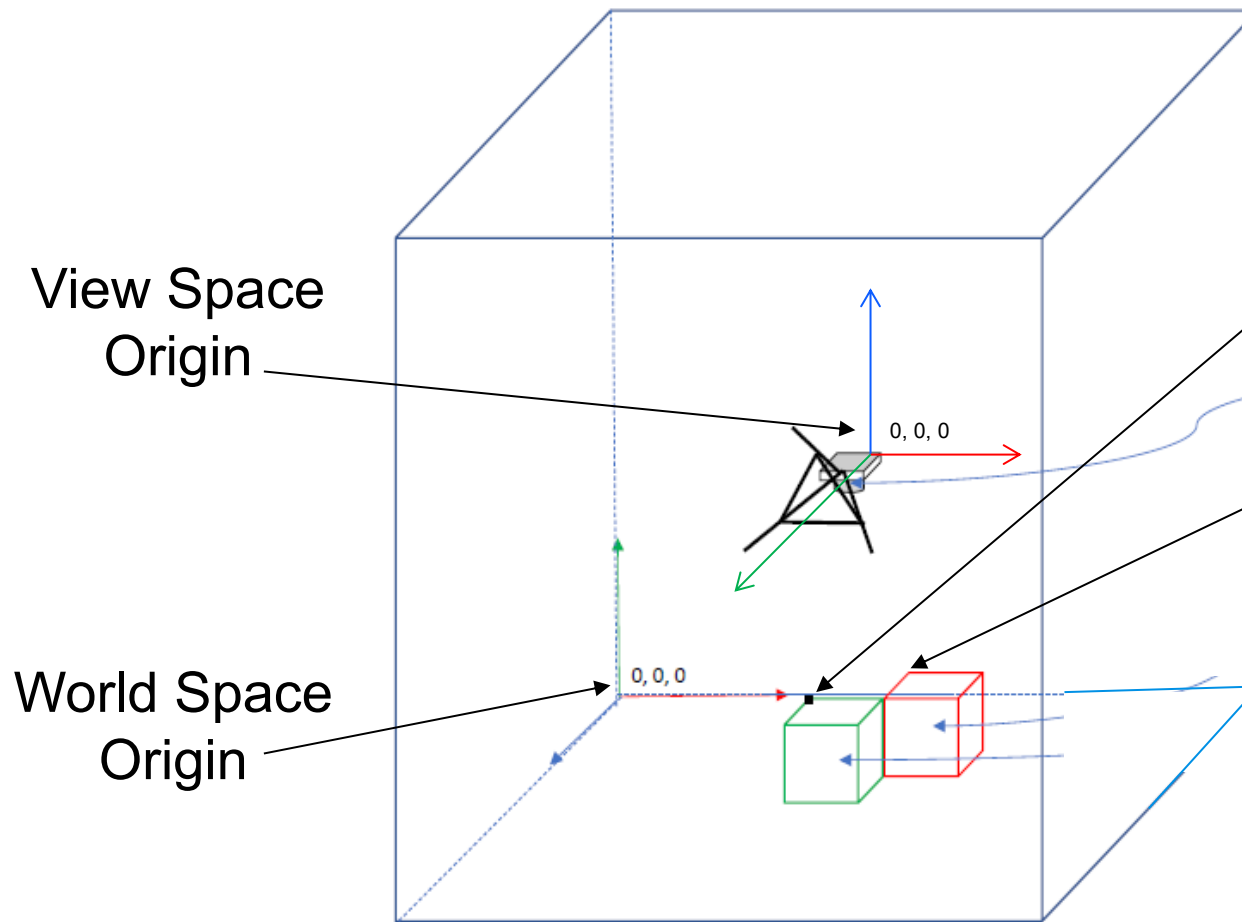
- ◆ The view space has each vertex from the point of view of the camera.

观察空间的原点在相机的位置处，坐标系朝向 -Z 轴。

- ◆ The origin of the view space is the location of the camera, pointing towards the negative Z axis.



View Space



$$\begin{pmatrix} 4 \\ 1 \\ 6 \end{pmatrix} + \begin{pmatrix} -5 \\ -5 \\ -5 \end{pmatrix} = \begin{pmatrix} -1 \\ -4 \\ 1 \end{pmatrix}$$

$$V1_{\text{world}} = (4, 1, 6)$$

$$V1_{\text{view}} = (-1, -4, 1)$$

$$\text{Camera Position} = (5, 5, 5)$$

$$V1_{\text{world}} = (6, 1, 4)$$

$$V1_{\text{view}} = ? \quad (1, -4, -1)$$

World Space -> View Space

View Matrix

观察矩阵会把所有顶点从世界空间转换到观察空间。

- A view matrix transforms all vertices from world space to view space.

- Example

- We want camera at position (5, 5, 5)

We want camera at position (5, 5, 5)

我们希望相机放在世界坐标 (5, 5, 5)。

思考点：

在世界空间里相机是在 (5,5,5)，

但在**相机空间 (View Space)**中，相机应该在 (0,0,0)。

因此我们要做的是：

“把相机放到原点”，也就是把整个世界朝相机的反方向平移。

$$T = \begin{pmatrix} 1 & 0 & 0 & -5 \\ 0 & 1 & 0 & -5 \\ 0 & 0 & 1 & -5 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

NEGATED

- We want to rotate camera (-90°) around the x-axis.

$$Rx(90) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



World Space -> View Space

View Matrix

- $v1_{world} = \begin{pmatrix} 4 \\ 1 \\ 6 \\ 1 \end{pmatrix}$

- $u = T \times v1_{world} = \begin{pmatrix} 1 & 0 & 0 & -5 \\ 0 & 1 & 0 & -5 \\ 0 & 0 & 1 & -5 \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} 4 \\ 1 \\ 6 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -4 \\ 1 \\ 1 \end{pmatrix}$

- $v1_{view} = Rx(1.57) \times u = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} -1 \\ -4 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ -4 \\ 1 \end{pmatrix}$
从右向左：先平移再旋转

Assessment 1 - In-class Quiz

An in-class quiz will be held this week:

Date: Wednesday, 5 November 2025,

Time: 2:00pm - 2:30pm

Venue: IAMET-406

Sample questions and a practice worksheet is available on Moodle.

Clip Space 裁剪空间

裁剪空间是一种坐标系统，用来判断哪些顶点在视野内（可见），哪些顶点不可见（被裁剪掉），最终只保留可见顶点进入下一阶段绘制。

- ◆ Clip space is a coordinate system that is used to determine which vertices are visible and should be included in the final image.

裁剪空间是顶点经过投影变换（Projection）后的中间空间，在进入屏幕绘制之前的一步。

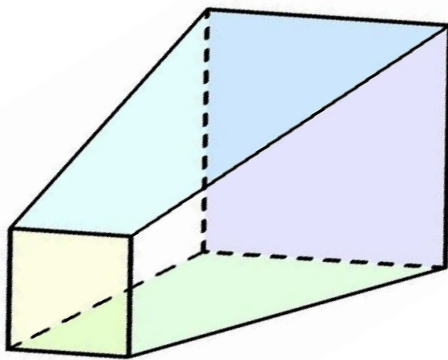
- ◆ Clip space is an intermediate space after vertices have been projected to what is going to be drawn to the screen



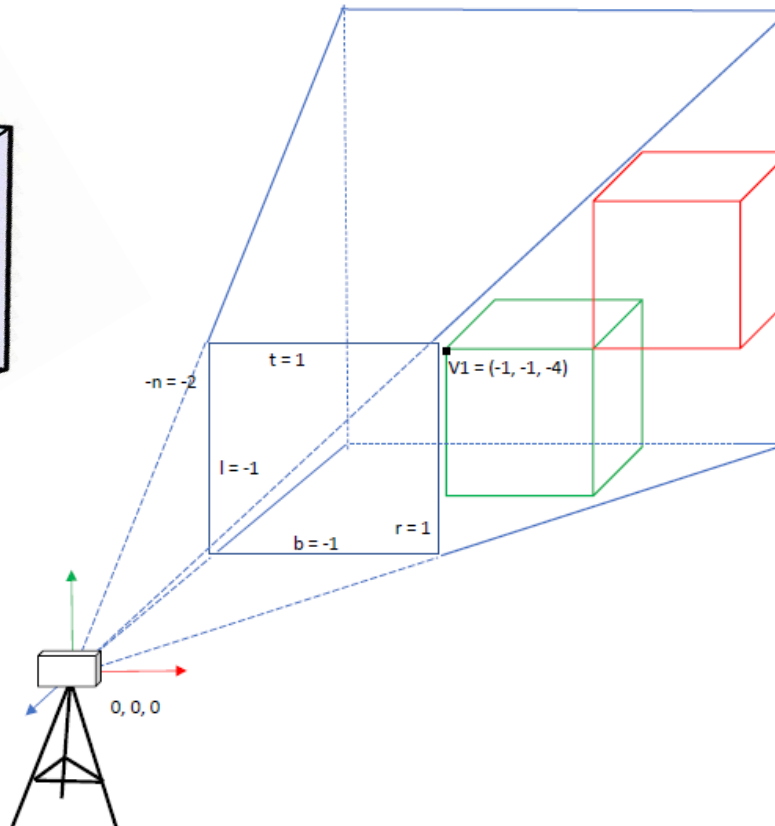
Clip Space

一个视锥体有“近裁剪面 (near plane)”和“远裁剪面 (far plane)”。

- ◆ **A frustum has a near plane and a far plane**
视锥体由六个边界参数定义：左 (l)、右 (r)、上 (t)、下 (b)、近 (n)、远 (f)。
- ◆ **A frustum is defined using left (l), right (r), top (t), bottom (b), near (n) and far (f) values**



Frustum



名称	意义	数学符号
Left Plane	左边界	$x = l$
Right Plane	右边界	$x = r$
Top Plane	上边界	$y = t$
Bottom Plane	下边界	$y = b$
Near Plane	最近裁剪面	$z = -n$
Far Plane	最远裁剪面	$z = -f$

右侧大图分析：

- 相机在 $(0,0,0)$ (视图原点)；
- 朝向 负 Z 方向；
- 可以看到相机前面有两个立方体 (绿色近，红色远)。

标注含义：

- $l = -1, r = 1 \rightarrow$ 左右边界；
- $b = -1, t = 1 \rightarrow$ 上下边界；
- $n = 2, f = 6 \rightarrow$ 近裁剪面与远裁剪面；
- 顶点 $V1 = (-1, -1, -4) \rightarrow$ 在视锥体内部，可见；
- 红色方块处于 farther plane ($z = -6$)，绿色方块在 nearer plane ($z = -4$)。

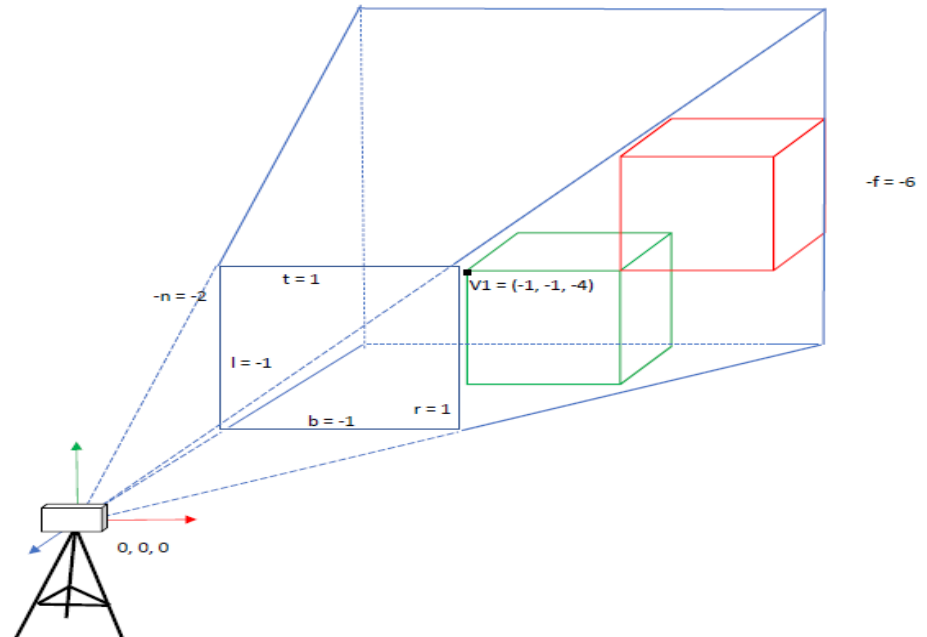
总体来说：

只有位于这个视锥体内部的物体，才会被保留到 Clip Space；
其余部分在进入 GPU 下一阶段前都会被丢弃。

View Space -> Clip Space Projection Matrix

$$P = \begin{pmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2(f*n)}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

矩阵的最后一行就是W分量



n – The distance from the viewer to the near plane (= 2)

f – The distance from the viewer to the far plane (= 6)

r – The distance of the right (or left) edge from the centre of the near plane (= 1)

t – The distance of the top (or bottom) edge from the centre of the near plane (= 1)

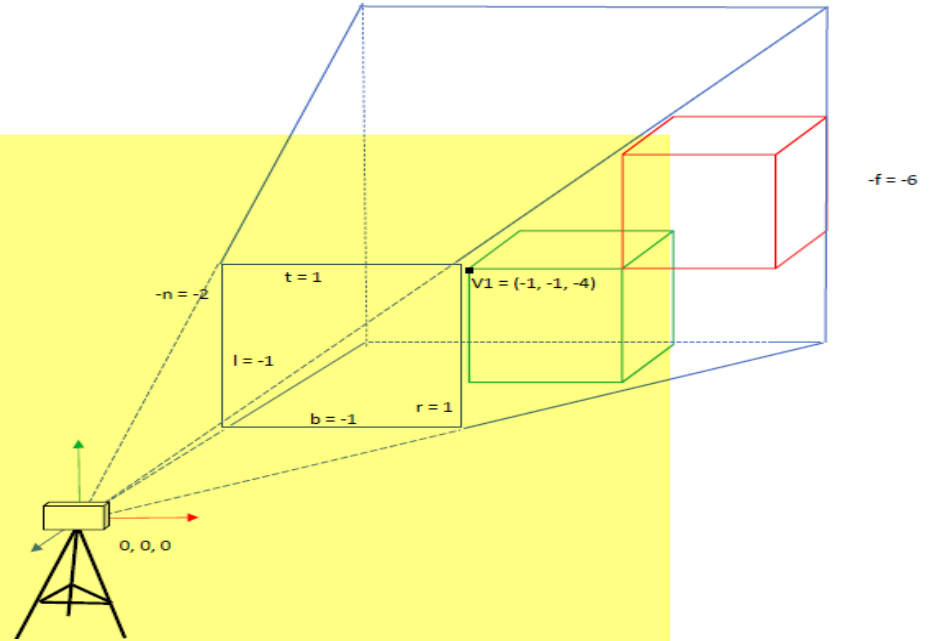
→ 对称情况下:

左 (l) = $-r$, 下 (b) = $-t$

视锥体是一个中心对称的“金字塔体”。

View Space -> Clip Space Projection Matrix

$$\blacksquare P = \begin{pmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & \frac{-(f+n)}{f-n} & \frac{-2(f*n)}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$



$$\blacksquare P = \begin{pmatrix} \frac{2}{1} & 0 & 0 & 0 \\ 0 & \frac{2}{1} & 0 & 0 \\ 0 & 0 & \frac{-(6+2)}{6-2} & \frac{-2(6*2)}{6-2} \\ 0 & 0 & -1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

View Space -> Clip Space

Projection Matrix

从观察空间到裁剪空间的转换，是通过投影矩阵实现的。

- ◆ The transformation from view space to clip space is achieved by using a projection matrix.

$$V_{clip} = P * V_{view}$$

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix} \times \begin{pmatrix} -1 \\ -1 \\ -4 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 * -1 + 0 * -1 + 0 * -4 + 0 * 1 \\ 0 * -1 + 2 * -1 + 0 * -4 + 0 * 1 \\ 0 * -1 + 0 * -1 + -2 * -4 + -6 * 1 \\ 0 * -1 + 0 * -1 + -1 * -4 + 0 * 1 \end{pmatrix} = \begin{pmatrix} -2 \\ -2 \\ 2 \\ 4 \end{pmatrix}$$

如果一个顶点在裁剪空间中超出范围（即 x, y, z 任意一个分量超出 [-w, w]），它会被裁剪掉。

- ◆ If a vertex lies outside the frustum (e.g., x, y or z component lies outside the [-w, w] range), it is clipped.

这里 $v_{clip} = (-2, -2, 2, 4)$

矩阵的最后一行就是W分量，so W=4

判断是否在范围内：

- x = -2 → 在 [-4, 4] 内 ✓
- y = -2 → 在 [-4, 4] 内 ✓
- z = 2 → 在 [-4, 4] 内 ✓

✓ 该点在视锥体内，不被裁剪。

NDC Space

NDC Space (Normalized Device Coordinates)
标准化设备坐标空间

无论屏幕是 800×600 还是 4K 分辨率,

NDC 中 $(x, y) = (1, 1)$ 总表示屏幕右上角,

而 $(0, 0)$ 表示屏幕中心。

NDC 空间 (标准化设备坐标) 是一种与屏幕无关的显示坐标系统。

- ◆ Normalized Device Coordinate or NDC space is a screen independent display coordinate system

NDC 空间在 x 、 y 、 z 三个维度上的范围都是从 -1 到 1。

- ◆ It has a range of -1 to 1 , in x y and z dimensions

NDC 空间的原点位于中心 $(0, 0, 0)$ 。

- ◆ The origin of NDC space is at the centre

- $(0,0)$ 是屏幕的中点;
- 正 $X \rightarrow$ 向右;
- 正 $Y \rightarrow$ 向上;
- 正 $Z \rightarrow$ 朝屏幕外 (远离相机)。

坐标轴	范围	对应位置
$x \in [-1, 1]$	水平方向	-1 是屏幕左边缘, 1 是右边缘
$y \in [-1, 1]$	垂直方向	-1 是底部, 1 是顶部
$z \in [-1, 1]$	深度方向	-1 是近平面, 1 是远平面 (OpenGL 定义)



注意:

不同 API 对 z 的范围定义不同:

- OpenGL: $z \in [-1, 1]$

- DirectX / Vulkan: $z \in [0, 1]$

NDC Space

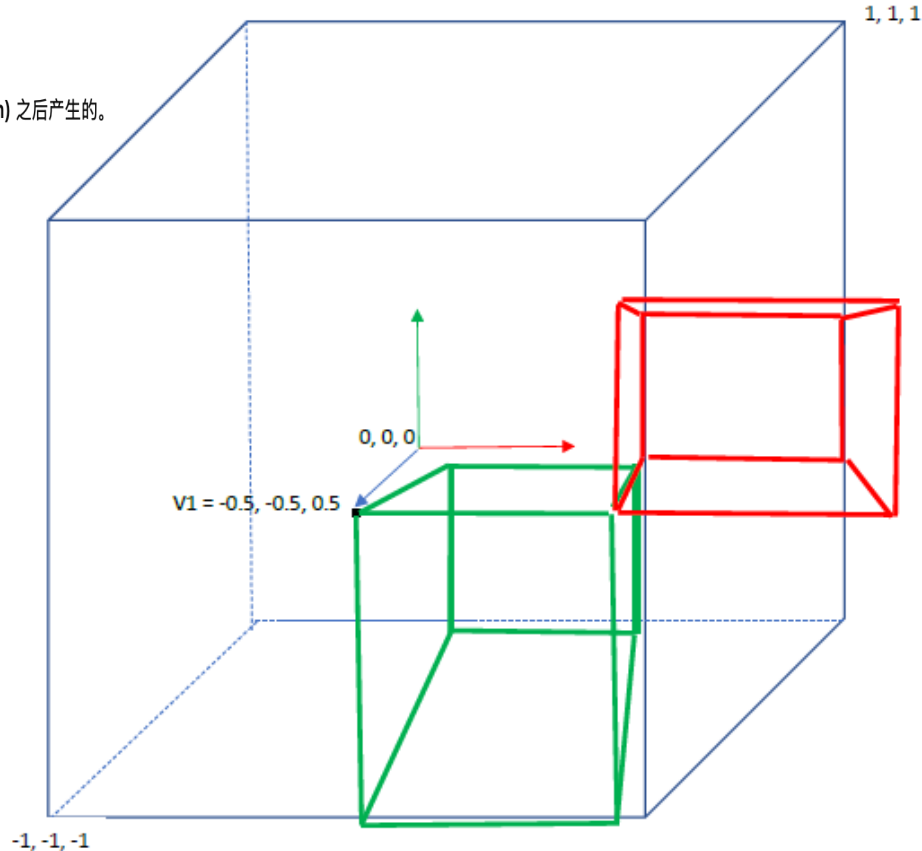
注意现在红色和绿色立方体都有了透视效果：它们的远面比近面看起来更小。

- ◆ Notice that the red and green cubes now have perspective, and the far faces of the cubes are smaller than the near faces.

- 这正是透视投影 (Perspective Projection) 的视觉特征：

- 物体距离相机越远，在投影后的空间中看起来越小。

- 这种“近大远小”的效果就是在 透视除法 (Perspective Division) 之后产生的。



Clip Space -> NDC Space

Perspective Division

从裁剪空间到 NDC 空间的变换，是通过透视除法（也叫齐次化）实现的。

- ◆ The transformation from clip space to NDC space is achieved by perspective division, also called homogenisation

$$NDC \begin{pmatrix} \frac{CLIP_x}{CLIP_w} \\ \frac{CLIP_y}{CLIP_w} \\ \frac{CLIP_z}{CLIP_w} \\ \frac{CLIP_w}{CLIP_w} \end{pmatrix} \quad v1_{CLIP} = \begin{pmatrix} -2 \\ -2 \\ 2 \\ 4 \end{pmatrix} \Rightarrow v1_{NDC} = \begin{pmatrix} -\frac{2}{4} \\ -\frac{2}{4} \\ \frac{2}{4} \\ \frac{4}{4} \end{pmatrix} = \begin{pmatrix} -0.5 \\ -0.5 \\ 0.5 \\ 1 \end{pmatrix}$$

Projection Matrix

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

两点之间在 View Space (观察空间) 中的 x 坐标差距为 2：
 $1 - (-1) = 2$

When z is -4, the distance
 between the two x coordinates
 is 2 in view space

Projection Matrix

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

View
Space
Point1

$$\begin{pmatrix} -1 \\ -1 \\ -4 \\ 1 \end{pmatrix}$$

Clip
Space
Point1

$$\begin{pmatrix} -2 \\ -2 \\ 2 \\ 4 \end{pmatrix}$$

两点在 Clip Space 中的 x 差距 = 4；
 而在 View Space 中的差距 = 2；
 也就是说经过投影矩阵后，它们的差距被“放大”成了 4。

View
Space
Point2

$$\begin{pmatrix} 1 \\ -1 \\ -4 \\ 1 \end{pmatrix}$$

Clip
Space
Point2

$$\begin{pmatrix} 2 \\ -2 \\ 2 \\ 4 \end{pmatrix}$$

Projection Matrix

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

×

View Space Point1

$$\begin{pmatrix} -1 \\ -1 \\ -6 \\ 1 \end{pmatrix}$$

=

Clip Space Point1

$$\begin{pmatrix} -2 \\ -2 \\ 6 \\ 6 \end{pmatrix}$$

两点之间在 View Space (观察空间) 中的 x 坐标差距为 2：
 $1 - (-1) = 2$

When z is -6, the distance between the two x coordinates is still 2 in view space

Projection Matrix

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

×

View Space Point2

$$\begin{pmatrix} 1 \\ -1 \\ -6 \\ 1 \end{pmatrix}$$

=

Clip Space Point2

$$\begin{pmatrix} 2 \\ -2 \\ 6 \\ 6 \end{pmatrix}$$

透视投影通过除以 w 压缩坐标，让距离相机越远的物体投影越小。
在 NDC 空间中， x 、 y 用于屏幕映射， z 只用于深度测试。

Projection Matrix

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

$\times$

View
Space
Point1

$$\begin{pmatrix} -1 \\ -1 \\ -4 \\ 1 \end{pmatrix}$$

$=$

Clip
Space
Point1

$$\begin{pmatrix} -2 \\ -2 \\ 2 \\ 4 \end{pmatrix}$$

$=$

NDC
Space
Point1

$$\begin{pmatrix} -0.5 \\ -0.5 \\ 0.5 \\ 1 \end{pmatrix}$$

When z is -4 , the distance between the two x coordinates is 1 in NDC space

在世界中这两个点相隔 2 个单位；
但经过投影后（除以 $w=4$ ），它们的投影宽度仅剩 1 单位；
这说明：相同大小的物体，在距离 $z=-4$ 时会被缩小一半。

Projection Matrix

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

$\times$

View
Space
Point2

$$\begin{pmatrix} 1 \\ -1 \\ -4 \\ 1 \end{pmatrix}$$

$=$

Clip
Space
Point2

$$\begin{pmatrix} 2 \\ -2 \\ 2 \\ 4 \end{pmatrix}$$

$=$

NDC
Space
Point2

$$\begin{pmatrix} 0.5 \\ -0.5 \\ 0.5 \\ 1 \end{pmatrix}$$

In NDC space, z -coordinate is no longer meaningful. It is only used for depth-test.

在 NDC 空间中， z 坐标不再用于视觉显示，只用于深度测试（Depth Test）。

Projection Matrix

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

×

View
Space
Point1

$$\begin{pmatrix} -1 \\ -1 \\ -6 \\ 1 \end{pmatrix}$$

=

Clip
Space
Point1

$$\begin{pmatrix} -2 \\ -2 \\ 6 \\ 6 \end{pmatrix}$$

=

NDC
Space
Point1

$$\begin{pmatrix} -0.3 \\ -0.3 \\ 1 \\ 1 \end{pmatrix}$$

When z is -6, the distance between the two x coordinates is 0.6 (shorter than 1) in NDC space

当 z 从 -4 变成 -6 时, x 的范围从 ± 0.5 缩小到 ± 0.33 ;
这意味着同样大小的物体, 距离变远后, 在投影结果中看起来更“窄” ;
这正是人眼熟悉的透视现象。
物体的真实宽度 (x 距离) 没有变 ;
但由于更远, 它的投影距离缩短为 0.6 (从 1 缩小了 40%) ;
所以: 越远, 投影越小, 视觉上“变小”。

Projection Matrix

$$\begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -2 & -6 \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

×

View
Space
Point2

$$\begin{pmatrix} 1 \\ -1 \\ -6 \\ 1 \end{pmatrix}$$

=

Clip
Space
Point2

$$\begin{pmatrix} 2 \\ -2 \\ 6 \\ 6 \end{pmatrix}$$

=

NDC
Space
Point2

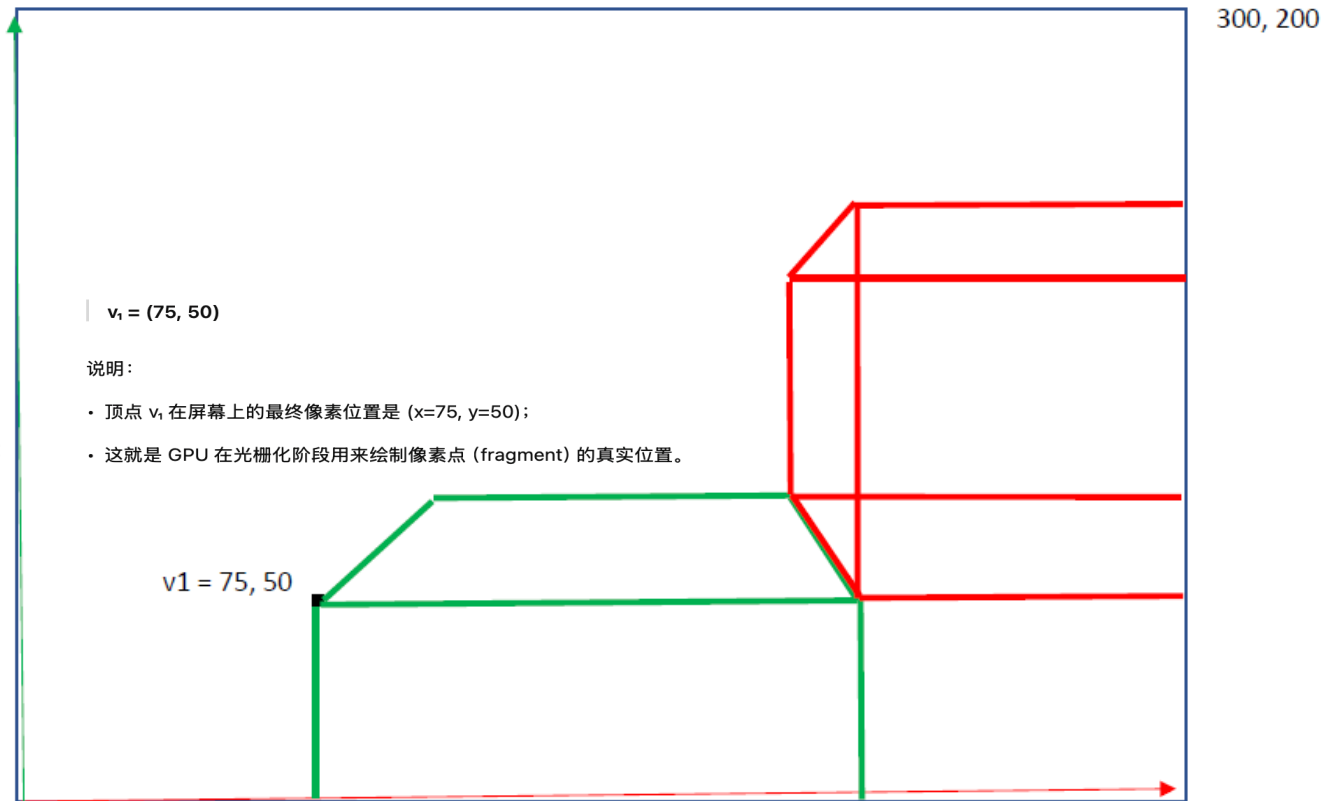
$$\begin{pmatrix} 0.3 \\ -0.3 \\ 1 \\ 1 \end{pmatrix}$$

In NDC space, z-coordinate is no longer meaningful. It is only used for depth-test.

Screen Space

屏幕空间与屏幕上的像素位置一一对应。

- ◆ The screen space matches the pixels on the screen
所有之前的坐标（Model、World、View、Clip、NDC）都属于数学坐标系；
只有在 Screen Space，坐标才与**实际像素位置（Pixel Coordinates）**相匹配。



0, 0



The University of
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

NDC Space -> Screen Space

Viewport Transformation

NDC 空间中的顶点通过视口变换被映射到屏幕空间。

- ◆ The vertices in NDC space are transformed to screen space using the viewport transformation
在屏幕空间中的顶点用于构建三角形，然后执行光栅化（Rasterisation）。
- ◆ Vertices in screen space are then used to construct triangles, and rasterisation is performed

NDC
Space
Point

$$\begin{bmatrix} -0.5 \\ -0.5 \\ 0.5 \\ 1 \end{bmatrix}$$

$$screenx = (NDCx + 1.0) * (width/2) \quad \text{原理:}$$

$$screeny = (NDCy + 1.0) * (height/2)$$

$$x = (-0.5 + 1.0) * \left(\frac{300}{2}\right) = 75$$

$$y = (-0.5 + 1.0) * \left(\frac{200}{2}\right) = 50$$

- +1.0 是为了把范围从 [-1,1] 移到 [0,2];
- 再乘以一半的屏幕宽/高 (width/2, height/2);
- 最终范围变为 [0, width] × [0, height];
- 即: 左下角 (-1,-1) → (0,0), 右上角 (1,1) → (width, height)。

In screen space, z-coordinate is not represented any more.

在屏幕空间中，z 坐标不再用于表示位置。



Clip Space & NDC Space & Screen Space

- ◆ Clip space is an intermediate space after vertices have been projected to what is going to be drawn to the screen 裁剪空间是顶点经过投影后进入的中间空间，用来确定哪些部分将被绘制到屏幕上。

NDC 空间几乎等同于屏幕上的像素位置，处于此空间内的顶点会被渲染到对应的位置。

- ◆ NDC Space is almost identical to the pixels on the screen and vertices inside the NDC Space will be rendered at those positions

屏幕空间直接对应屏幕像素；顶点在此空间用于构建三角形，光栅化后像素被着色显示。

- ◆ Screen Space maps directly to the pixels on the screen, where vertices can be used to construct triangles, which are rasterised and the appropriate pixels are coloured accordingly



References

- ◆ *COMP3069 Lecture notes: Chapter 6 - Graphics Programming, OpenGL & GLSL*
- ◆ *Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL, Chapter 4*